Final Project · Introduction to Data Science

A Simple Recommender System with Association Rules (Apriori)

Group 4

Femi Novia Lina · Cesilia Natasya Nainggolan · Alya Fitri Nurhaliza

Data science methodology

We followed an 8-stage data science cycle, from business question to feedback.

01
Business understanding

02
Analytic approach

03
Data requirements

04
Data collection

05
Data understanding & preparation

06
Modelling

07
Evaluation & deployment

08
Feedback

1 · Business understanding

Goal

Help shoppers narrow down their choices by suggesting products they are likely to want.

Objective

Build a model that can power a recommendation feature: given the items already chosen, return relevant suggestions.

2 · Analytic approach

Predictive analytics

Learn from past baskets which items tend to be bought together, and predict what a customer may add next.

Prescriptive analytics

Turn that knowledge into an action: show the customer a short list of suggested products.

Method: **association rule mining** with the **Apriori** algorithm.

3–4 · Data requirements & collection

What we need: a list of transactions, each one the set of items one customer bought together.

Dataset: Grocery Products Purchase (Kaggle, AlexMiles). It is the *Groceries* dataset from the R package `arules`: one month of point-of-sale data from a local grocery store.

Source: Hahsler, Hornik & Reutterer (2006), *Implications of probabilistic data modeling for mining association rules*.

9,835
baskets

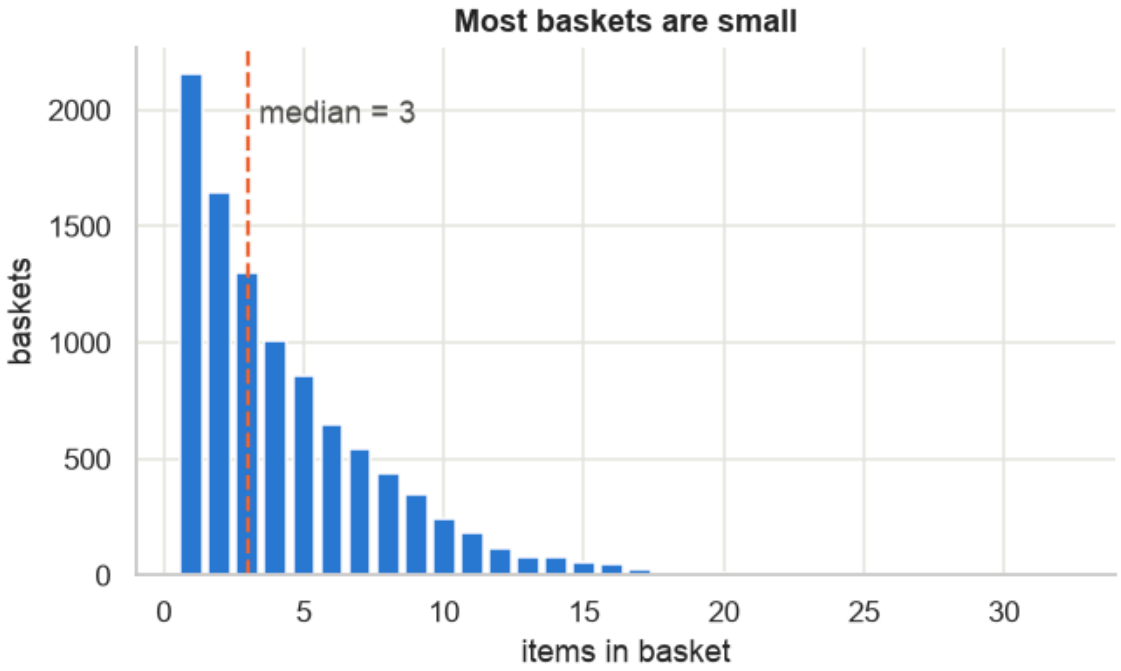
169
product categories

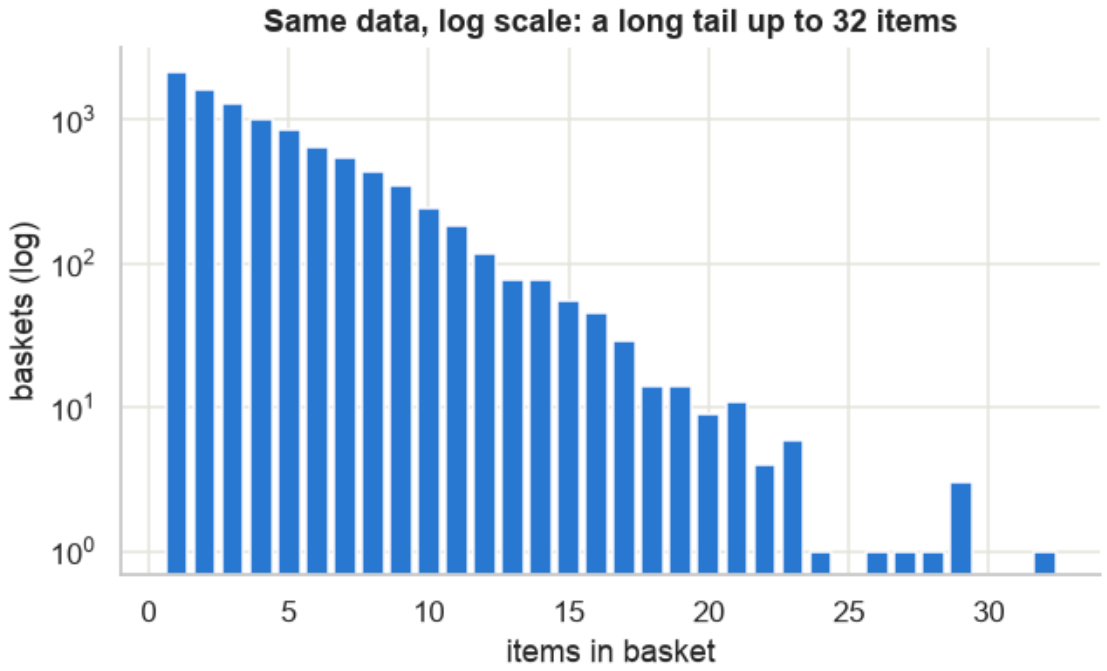
4.4
items per basket (average)

22%
baskets with a single item

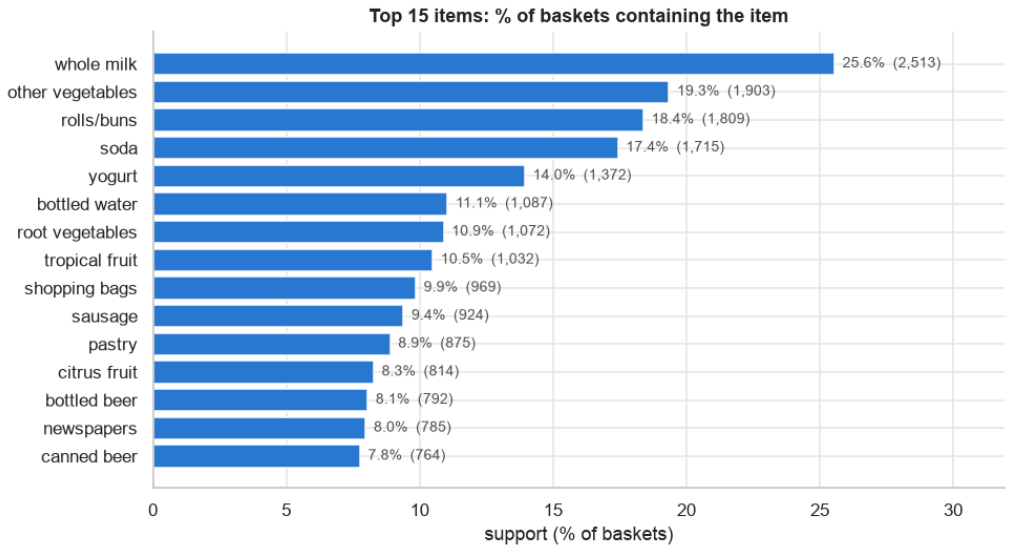
5 · Data understanding

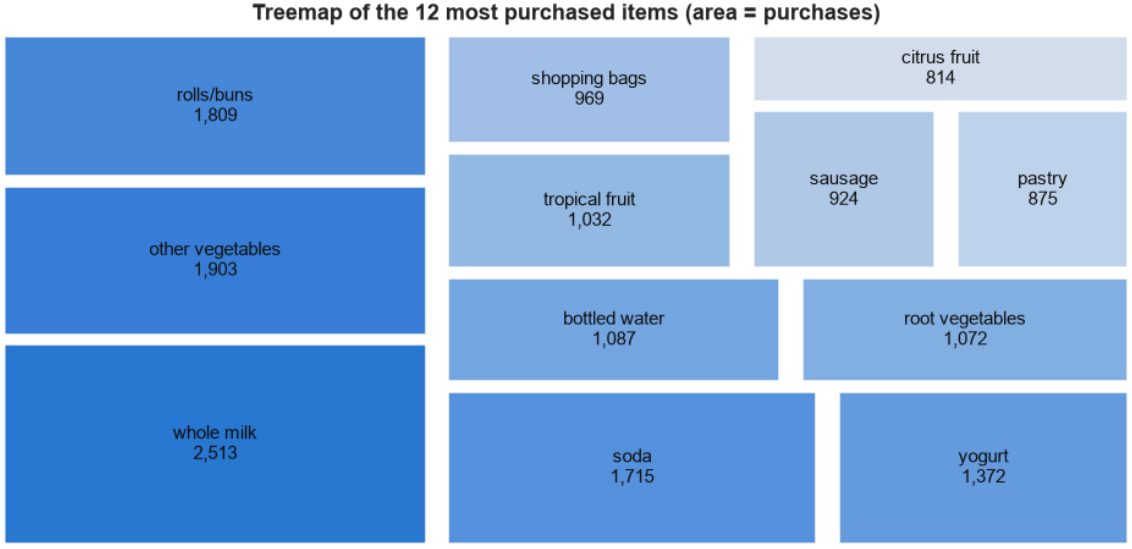
- **9,835 rows × 32 columns.** One row = one basket; columns are item *slots*, all text.
- **86% of cells are empty** (271,353 of 314,720). That is just the wide format, so no imputation is needed.
- **2,824 identical baskets** are real repeat purchases, so we keep them.
- **22% of baskets hold a single item.** The median basket has 3 items.





5 · EDA: top items

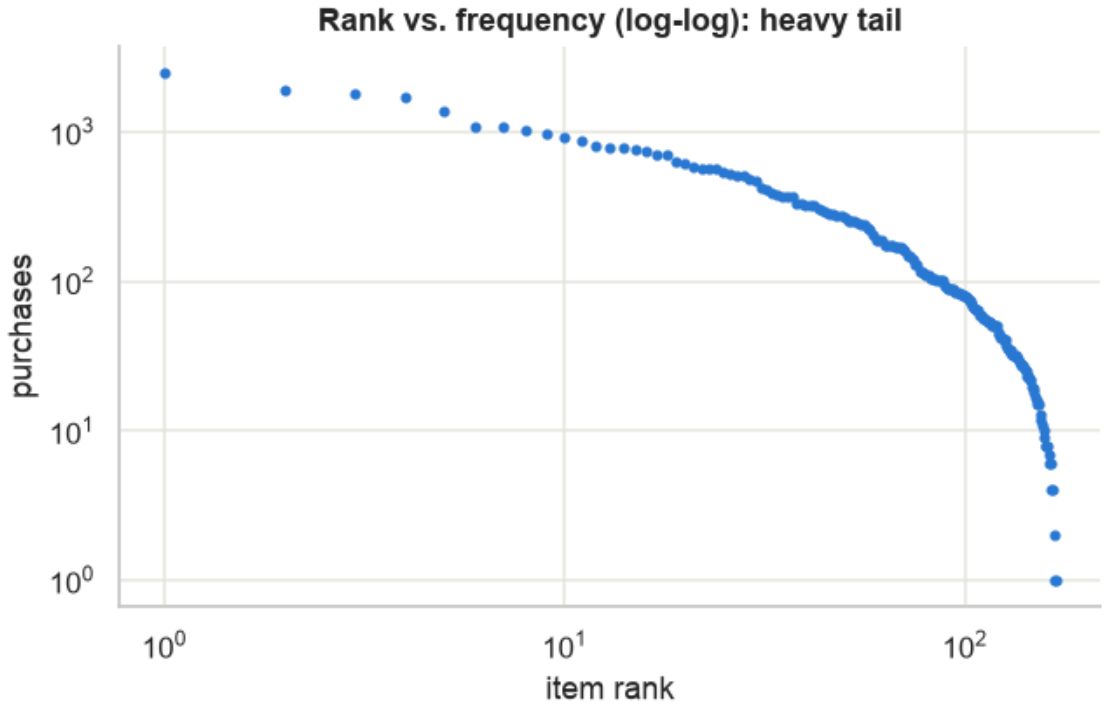




5 · EDA: the long tail

- **20 of 169 items (12%)** make up half of all purchases; 54 items make up 80%.
- 81 items appear in fewer than 1% of baskets, so they cannot show up in a rule at 1% minimum support. Apriori therefore favours popular items.





6 · Data preparation

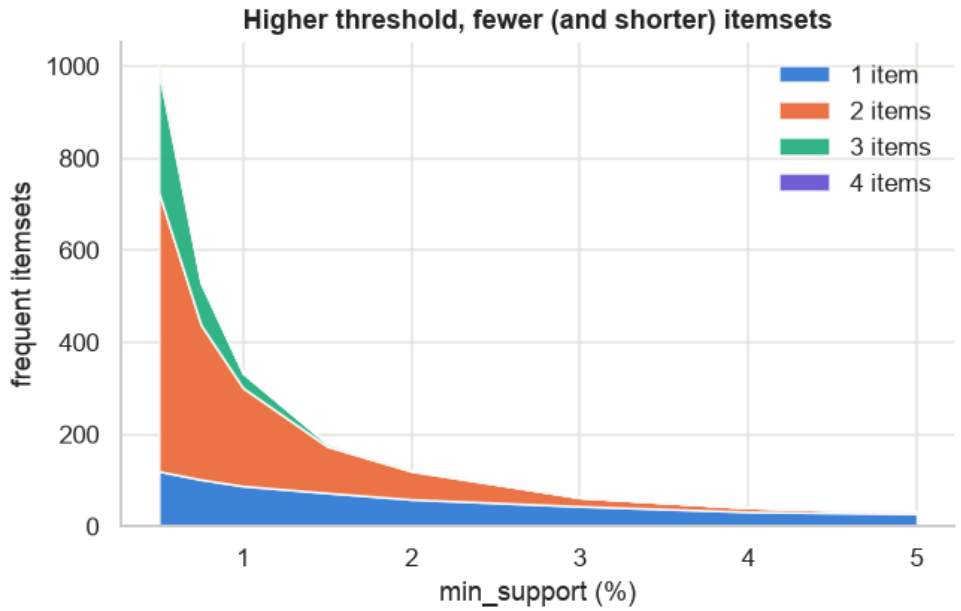
1. **Case folding:** lowercase and trim all item names.
2. **Rows to baskets:** turn each row into a list of items (drop the empty cells).
3. **Check consistency:** count each item to catch spelling variants (169 distinct items remain).
4. **Keep duplicate baskets:** they are genuine purchases, and removing them would understate popularity.
5. **One-hot encode** with `TransactionEncoder`: one True/False column per item (9,835 × 169).

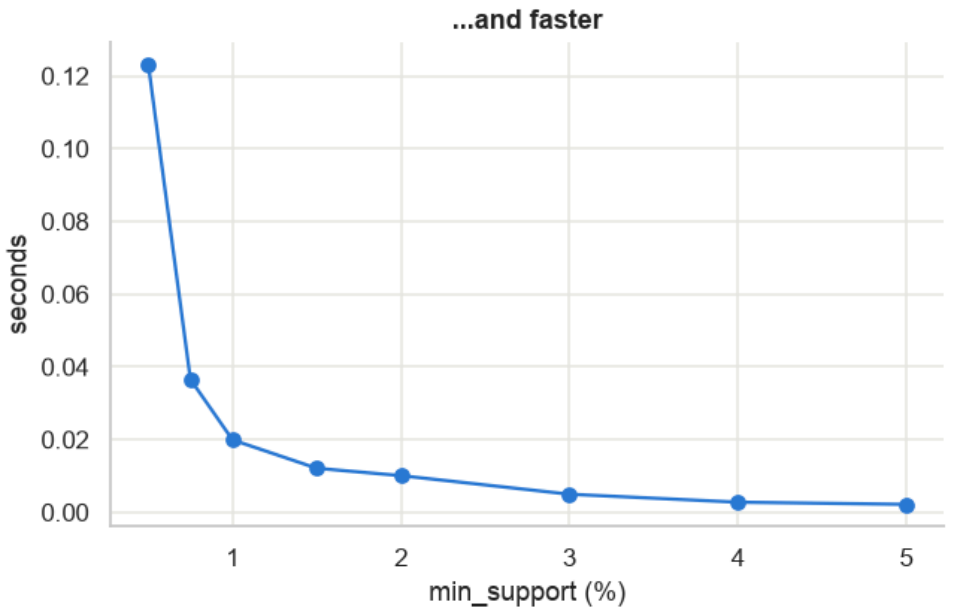
7 · Modelling: how Apriori works

An itemset is **frequent** if its **support** (share of baskets containing it) reaches `min_support`. Apriori builds itemsets level by level and prunes with one rule:

> **If an itemset is infrequent, every larger itemset containing it is infrequent too.**

We chose **min_support = 1%** (about 100 baskets): 333 frequent itemsets, enough pairs and triples to be useful. Above ~3% almost no pairs are left; below 1% rare combinations may be noise.





7 · From itemsets to rules

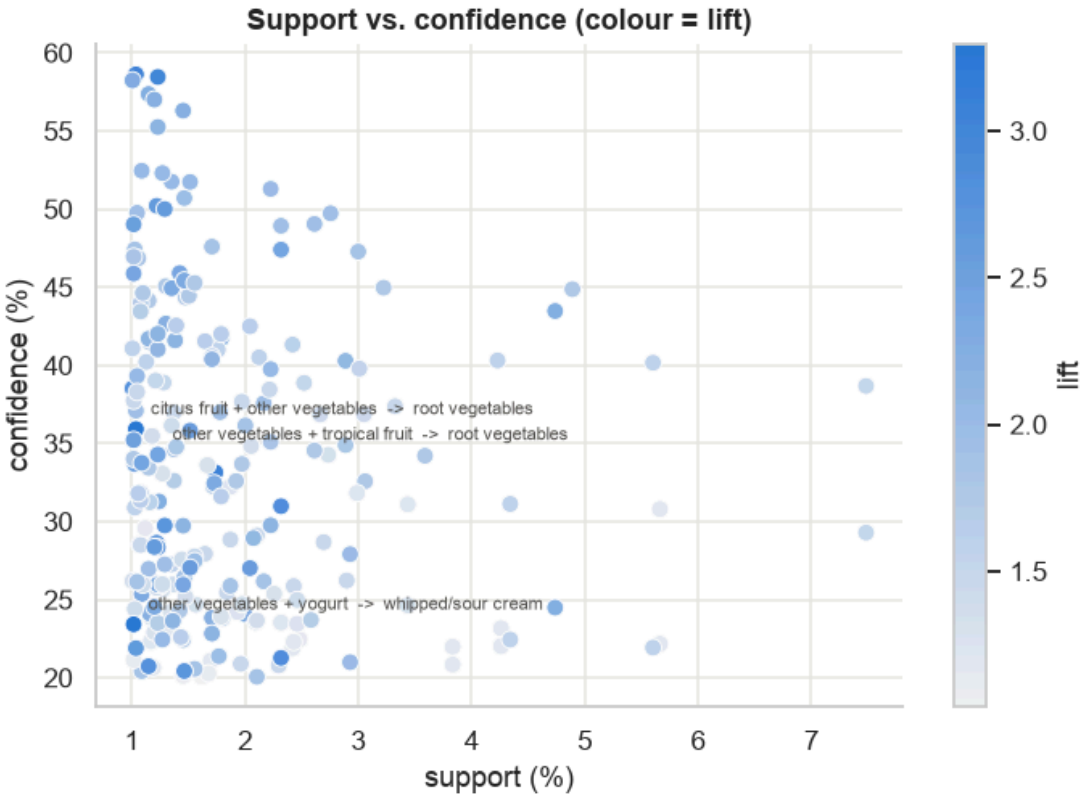
METRIC	FORMULA	MEANING
Support	P(A and B)	How common the combination is
Confidence	P(B | A)	How often B is bought when A is
Lift	confidence / P(B)	How much A raises the chance of B. 1 = no effect

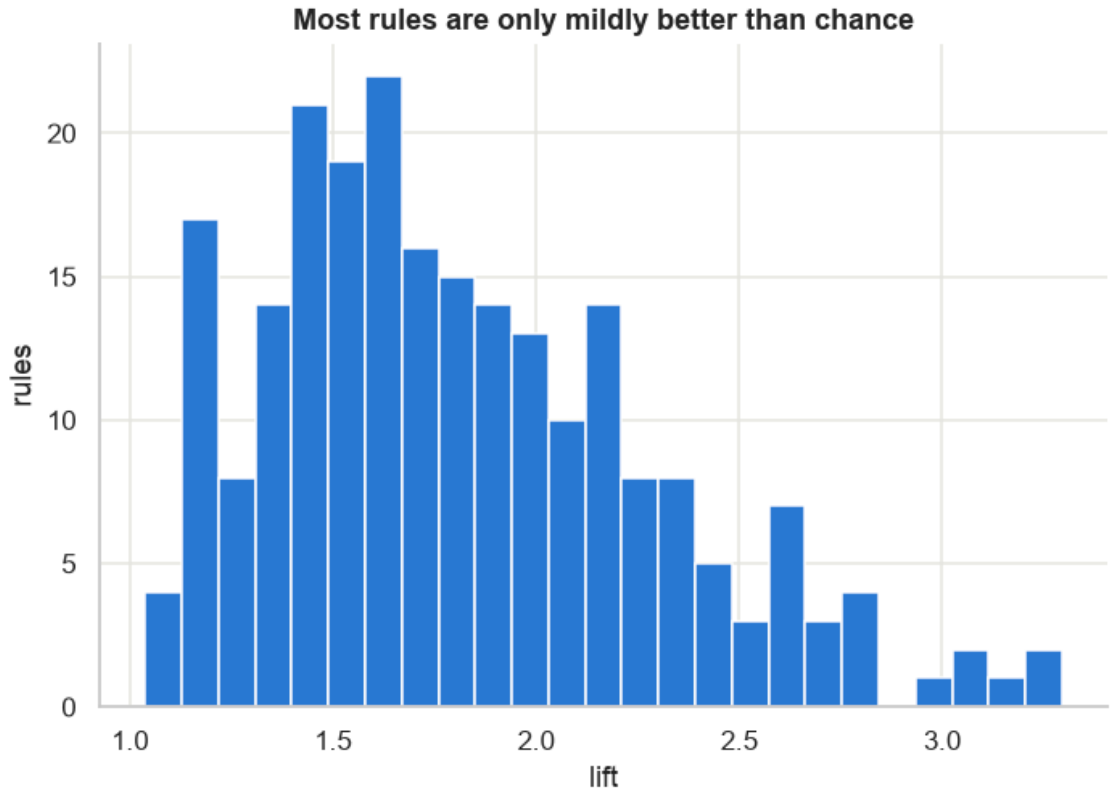
Why lift matters: `whole milk` is in 25% of baskets, so almost any rule ending in milk has high confidence. Lift removes that effect.

We kept **231 rules** with confidence ≥ 20% and lift > 1. (The original filter, confidence ≥ 1.5%, kept almost everything.)

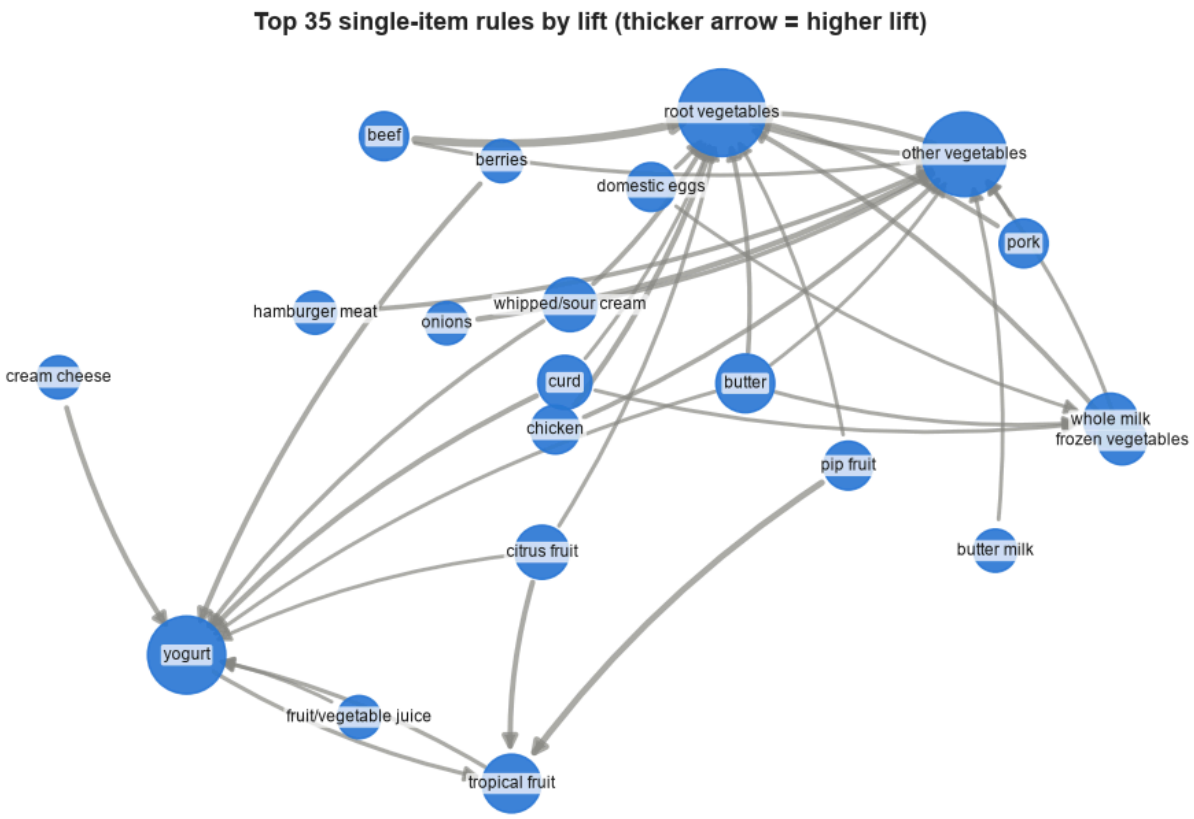
7 · What the rules look like

Dark points = high lift. The most frequent rules are not the most interesting ones; most rules are only mildly better than chance.





7 · The rule network



- Top 35 single-item rules by lift.
- Fresh produce dominates: `root vegetables`, `other vegetables`, `yogurt` and `tropical fruit` are the nodes many rules point to.
- Meat and dairy items (`beef`, `pork`, `chicken`, `butter`) mostly point into produce.

7 · Using the model

Customer's basket: `whole milk`

Original `find()`

Every item in any frequent itemset containing the basket, **unranked**:

45 items (beef, berries, bottled beer, bottled water, brown bread, butter, ... yogurt)

New `recommend()`

Ranked by rule confidence, top results:

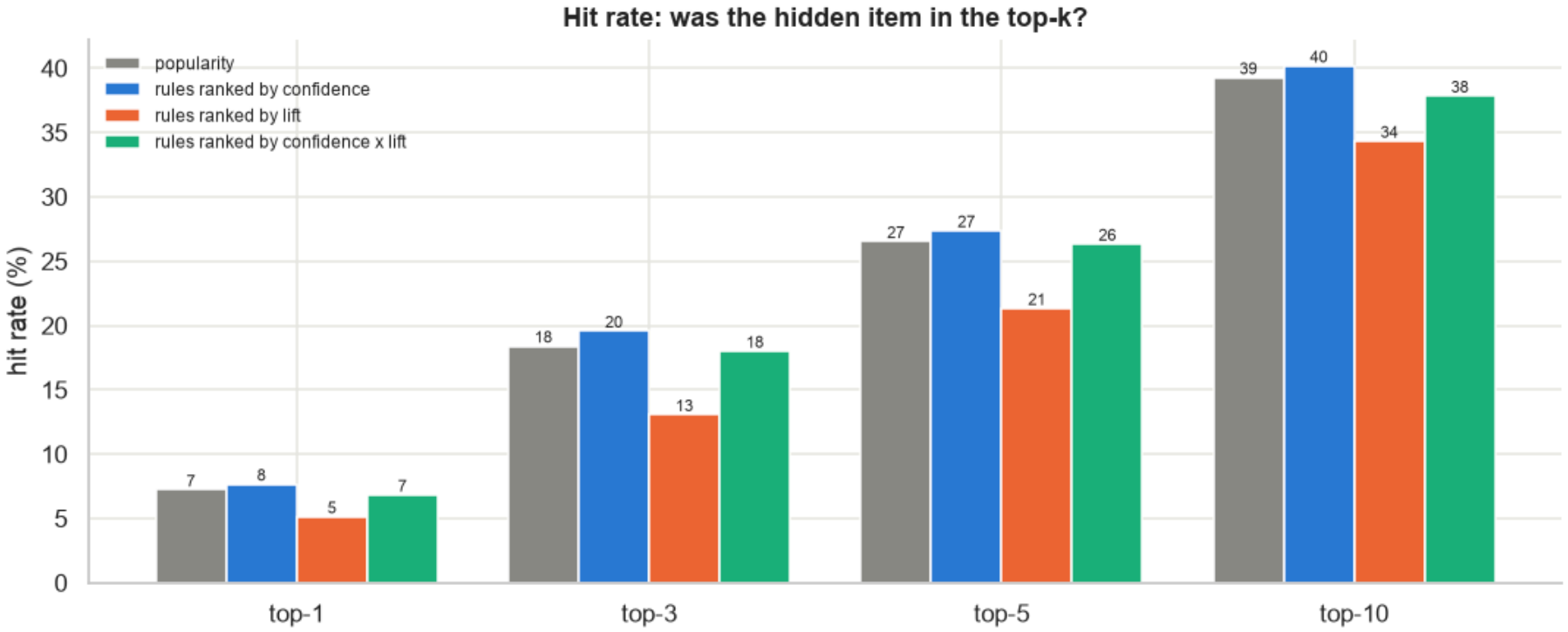
1. other vegetables **2.** rolls/buns **3.** yogurt

Basket `whole milk + root vegetables` → **other vegetables, yogurt, rolls/buns, tropical fruit**

The ranked version gives the customer a short, ordered list instead of an unordered pile.

8 · Evaluation

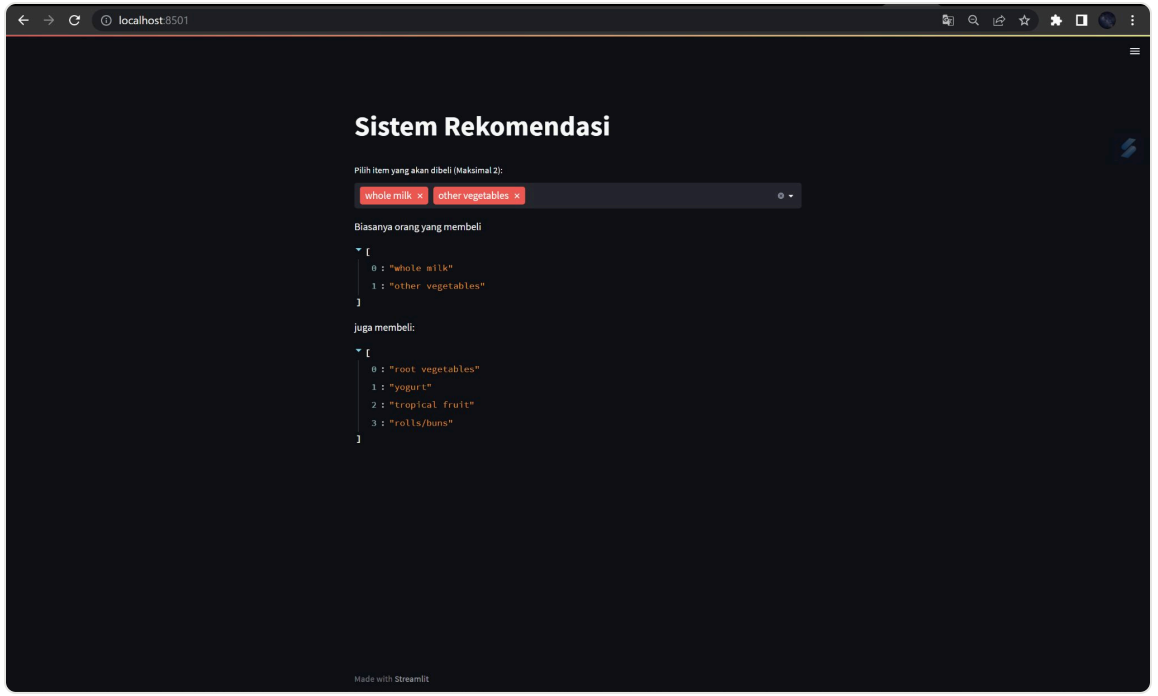
- **Test:** mine rules on 80% of baskets; on the other 20%, hide one item and check if it appears in the top-k.
- Ranked by **confidence**, rules are **on par with recommending best-sellers** (gap within sampling noise, about ±1 point).
- Ranking by **lift** is clearly worse for prediction: it favours rare items.
- Rules still offer something popularity cannot: **an explanation** ("because you bought X").



9 · Deployment

- The model is saved with `pickle` and loaded by a **Streamlit** app.
- The user picks up to **2 items** from a list (the longest itemset in the model has 3 items).
- The app calls `find()` and shows the suggestions.

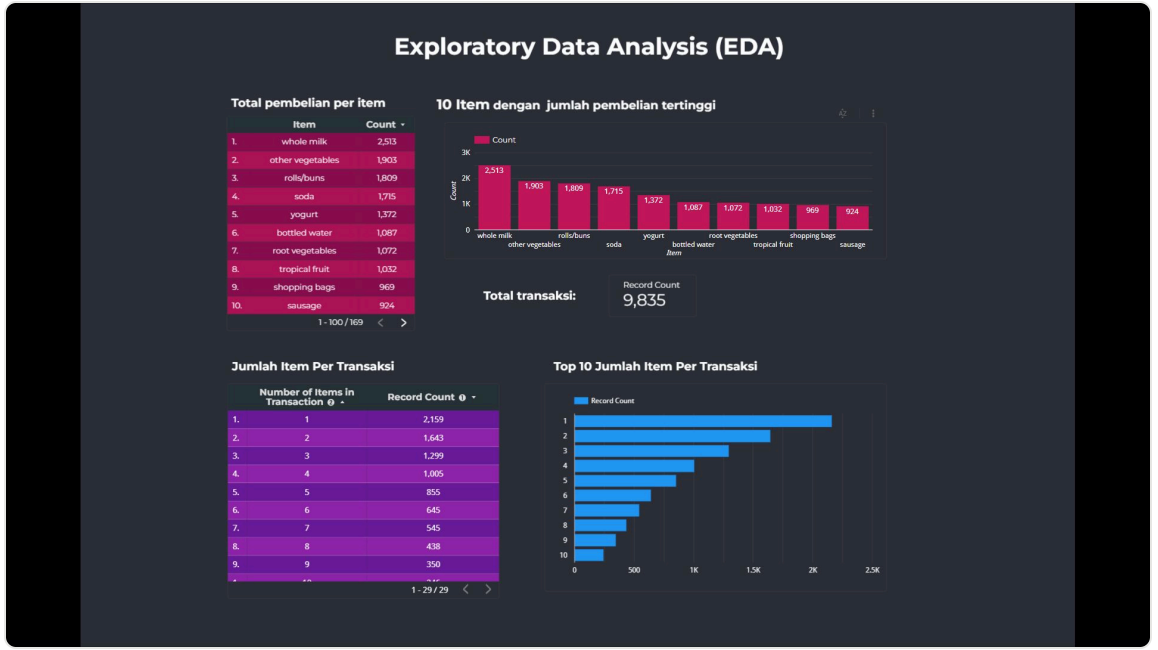
Live app (original Indonesian UI), hosted on Streamlit Cloud: feminovialina-simple-apriori-recommendation-system-d-app-93n7sm.streamlit.app



10 · Feedback

An **exploratory dashboard** lets stakeholders explore the data behind the model: purchases per item, the 10 most purchased items, total transactions (9,835) and the number of items per basket. Some chart titles are in Indonesian.

Built with Google Data Studio (now Looker Studio):
[Open the dashboard]



Limitations

Anonymous baskets

No customer history or taste profile, so no personalisation.

Popularity bias

Rare items almost never appear in rules.

Static & order-blind

Rules do not update on their own and ignore the order in which items were added.

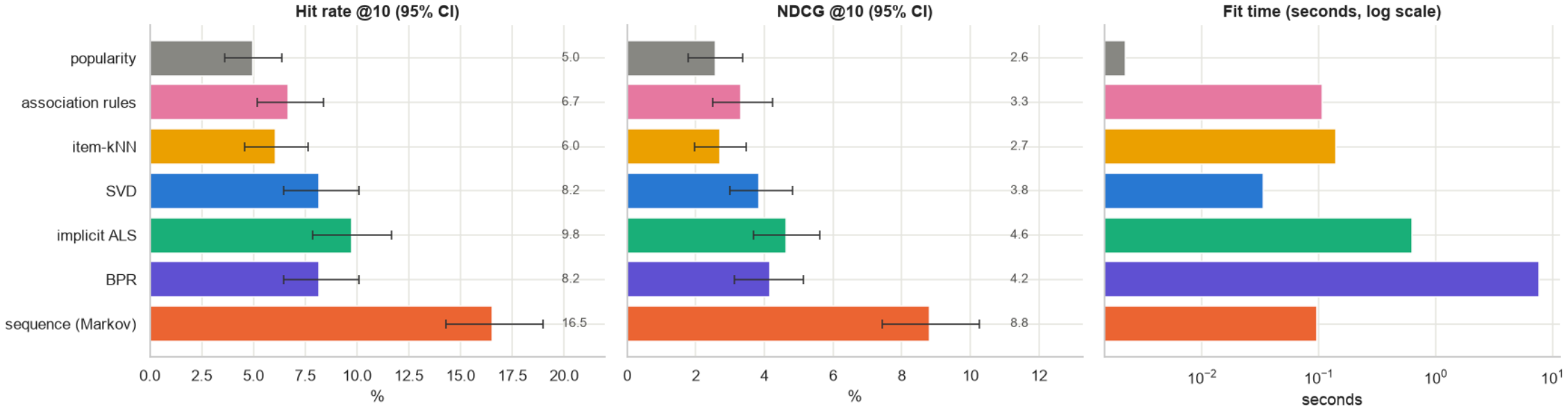
Cold start

A new product has no rules until it sells enough.

Next: how modern recommenders work

A companion notebook compares 7 recommenders on **MovieLens 100K** (943 users, 1,682 movies): popularity, association rules, item-kNN, SVD, implicit ALS, BPR and a sequence model. Learned models beat popularity only modestly.

Order matters most: a tiny Markov chain over what users watched next beat every non-sequential model (16.5% vs 9.8% hit rate @10). Real systems go further with two-tower networks, sequence transformers and generative retrieval.



Thank you!

Questions?

Femi Novia Lina · Cesilia Natasya Nainggolan · Alya Fitri Nurhaliza

Code, notebooks and data sources: see the project repository README.